

Lessa?! | لسه؟!

Smart Government Queue Tracking Platform

2-Day Hackathon Sprint Plan

Version 1.0 — Hackathon MVP Edition | June 2026

Team of 3 | 48 Hours | Version 1.0 MVP Delivery

1. Delivery Strategy

2-day hackathon sprint. Goal: a fully demo-able MVP with real data, functional queue operations, and an AI prediction showcase. The strategy is to work in three parallel workstreams (Backend, Frontend, and Data/DevOps/AI) with integration checkpoints at end of Day 1 and mid-morning Day 2.

2. Assumptions

- Team of 3: one backend-heavy (BE), one frontend-heavy (FE), one full-stack/devops (FS)
- React + Node/Express + PostgreSQL tech stack agreed before Day 1
- Development environment (Node, Postgres, Redis) configured before Day 1 begins
- Repository initialized with monorepo structure; CI not required for MVP
- Demo environment (Render or Railway) accessible and tested before Day 1
- Seed data script will be pre-written before demos begin

3. Critical Path

DB Schema → Backend Auth → Backend Booking → Backend Queue Engine → Integration Test → Supervisor + AI + Display Board → Demo Rehearsal

4. Team Assignments

Member	Role	Primary Responsibility
Engineer A	Backend	Auth, Booking Module, Queue Engine, Notifications, Queue Rules
Engineer B	Frontend	Citizen App, Staff Interface, Supervisor Dashboard
Engineer C	Full Stack + AI	DB migrations, Prediction Engine, Public Display Board, Reports, Seed Data, Deployment

5. Day 1 Plan — Hours 0–12

Day 1 goal: Core booking → queue → calling flow working end-to-end. All critical path stories complete.

Hour	Engineer A (Backend)	Engineer B (Frontend)	Engineer C (Full Stack + AI)
0–1	Finalize tech stack, set up repo, branch strategy	Same as BE	DB schema design + migration files
1–3	Auth module: register, OTP mock, JWT, RBAC middleware	React app scaffold: Vite + Tailwind + shadcn/ui, routing, i18n setup	Run migrations; seed: 1 branch, 3 services, 3 windows, 5 users

3–5	Service Catalog API: CRUD + public list with queue stats	Citizen: Registration + Login screens	Prediction Engine: heuristic formula + prediction API endpoint
5–6	Booking Module: POST /tickets (atomic sequence, validations)	Citizen: Service List page (public)	Public Display Board: page + API endpoint
6–7	Queue Engine: call-next, resolve, grace period timer	Citizen: Booking flow + Ticket confirmation screen	SSE infrastructure setup
7–8	Notification Module: create notifications on state changes	Citizen: My Ticket + real-time position display	Connect SSE to Frontend
8–9	Window Module: open/close/pause, staff-window assignment	Staff Interface: Window dashboard + Call Next button	Deploy to Render/Railway; verify environment
9–10	Integration testing: booking → queue → call → resolve	Staff Interface: Done/Skip/No-Show actions	Supervisor Dashboard: live window grid
10–11	Bug fixes from integration testing	Notification bell + in-app notification list	Reports API: daily-summary, peak-hours
11–12	Cancellation + modification flow; cancellation cutoff	Supervisor: Reports page + Announcement form	Admin Panel: user management + queue rules config

6. Day 2 Plan — Hours 13–24

Day 2 goal: Polish, QA, AI prediction, display board, demo prep, and final deployment.

Hour	Engineer A (Backend)	Engineer B (Frontend)	Engineer C (Full Stack + AI)
13–14	Bug fixes from Day 1 testing	Admin Panel: User creation + role assignment	Seed realistic demo data (100+ tickets across 2 hours)
14–15	Turn-approaching notification trigger	Public Display Board polish: large font, animations	Prediction snapshot storage (async)
15–16	Audit logging: complete coverage of all critical actions	Supervisor: Announcement posting + display	Historical factor computation from seeded data
16–17	End-to-end QA pass (all user flows, all roles)	Mobile responsiveness pass + RTL Arabic layout	QA: Arabic UI, RTL layout check, display board
17–18	Fix QA issues	Fix QA issues	Verify deployment; run load test (10 concurrent bookings)
18–19	Demo Prep: demo accounts, walk-through script	Demo polish: loading states, error messages, animations	Prepare demo video backup (screen recording)
19–20	Stretch: QR check-in mock or predicted wait countdown	Stretch: Animated ticket call display on display board	Stretch: Historical chart on reports page
20–21	Final demo rehearsal (all 3 together)	Final demo rehearsal	Final demo rehearsal
21–24	Buffer: remaining bugs, final deploy, presentation prep	Buffer	Buffer

7. Must / Should / Could Feature List

Priority	Feature	Owner
Must	Citizen registration + login	BE + FE
Must	Service list (public)	BE + FE

Must	Book a queue slot	BE + FE
Must	Staff: Call Next + resolve ticket	BE + FE
Must	Real-time ticket tracking (SSE or polling)	BE + FS + FE
Must	AI predicted wait time	FS
Must	Public display board	FS + FE
Must	Supervisor dashboard	FE + BE
Must	In-app notifications	BE + FE
Must	Arabic + English UI	FE
Should	Cancel/modify booking	BE + FE
Should	Turn-approaching notification	BE + FE
Should	Daily summary report	BE + FE
Should	Branch announcement	BE + FE
Should	Admin user management	BE + FE
Should	Audit logs (admin view)	BE + FE
Could	Predicted wait countdown animation	FE
Could	Animated display board transitions	FE
Could	Peak hours chart	FE
Could	Window performance report	BE
Could	Prediction snapshots + accuracy tracking	FS

8. Risk Mitigation

Risk	Mitigation
SSE too complex → use polling	Pre-decide: if SSE setup takes >1 hour, switch to 30-second polling
RTL layout breaks	Test Arabic UI at Hour 3 (not Hour 18); use CSS logical properties from the start
Queue race condition hard to debug	Write atomic DB query (SELECT FOR UPDATE) and test with 2 concurrent bookings early
Prediction gives nonsensical values	Clamp output to 0–240 minutes; show "N/A" if computation fails
Deploy fails on PaaS	Test deploy at end of Day 1; maintain fallback (localhost demo)
Seed data looks unrealistic	Pre-write seed script before Day 1; use realistic Egyptian names, Arabic service names

9. Testing Strategy

Level	What	When
-------	------	------

Unit	Prediction engine formula, ticket status transitions	Inline with dev; 30 min total
Integration	Booking flow end-to-end, concurrent booking (race condition), queue calling	End of Day 1
Manual QA	All 5 user roles, both languages, mobile viewport	Day 2 Hours 16–17
Load test	10 concurrent bookings against last slot	Day 2 Hour 17
Demo rehearsal	Full scenario walkthrough (all roles)	Day 2 Hours 20–21

10. Demo Preparation Checklist

Item	Details	Status
Admin account	phone: +201000000000	☐
Counter staff accounts	staff1, staff2 with window assignments	☐
Supervisor account	supervisor1 assigned to Cairo Central Branch	☐
Citizen accounts	5 accounts with realistic Egyptian names + Arabic variants	☐
Services seeded	NID (National ID), PASS (Passport), BC (Birth Certificate)	☐
Windows open	3 windows open and assigned to staff	☐
Historical tickets	50–80 tickets across services with mixed statuses (done, no-show, waiting)	☐
Live demo tickets	5 tickets in "booked/waiting" for live demo flow	☐
Branch announcement	Pre-drafted announcement ready to post during demo	☐
Arabic UI ready	Switch to AR locale; verify RTL layout	☐
Display board tab	Open on second screen/tab for demo	☐
Screen recording	Backup recording prepared in case of connectivity issues	☐
Demo script	Printed/rehearsed walkthrough covering all 5 roles	☐

11. Seed Data Reference

The following SQL snapshot captures the minimum seed data required for a convincing demo. Run this before any demo session.

```
-- Branch
INSERT INTO branches (id, name, name_ar, address) VALUES
('branch-01', 'Cairo Central Branch', 'أفرع القاهرة المركزي', '12 Tahrir Square, Cairo');
```

```

-- Services
INSERT INTO services (code, name_en, name_ar, avg_handling_time_minutes,
daily_capacity, branch_id) VALUES
  ('NID', 'National ID Renewal', 'تجديد بطاقة الرقم القومي', 5, 100, 'branch-01'),
  ('PASS', 'Passport Services', 'خدمات جواز السفر', 8, 80, 'branch-01'),
  ('BC', 'Birth Certificate', 'شهادة الميلاد', 4, 60, 'branch-01');

-- 80 historical tickets (yesterday, status: done) for AI historical factor
-- 5 active tickets today (status: waiting) for live demo
-- 1 announcement: "System working normally. Thank you for your patience."
```

12. Recommended Technical Defaults

These implementation defaults resolve known ambiguities before coding begins:

Decision	Recommended Default	Reasoning
OTP Provider	Mock: static code 123456, stored in env	Zero cost, zero complexity, demoable
Real-time mechanism	Start with polling (30s); add SSE for display board	Display board SSE is low risk; citizen polling is good enough
Redis	Optional; use JS Map in-memory for queue state	For single-server MVP with <200 concurrent users, JS Map is fine
Frontend structure	Single React app, role-based routing + layouts	Fastest to build; shared component library
Grace period timer	setTimeout + DB update for MVP	Simple; replace with pg-boss post-MVP
Check-in requirement	Optional; booked → waiting at position recalc	Reduces citizen friction
Queue position calc	Lazy calculation on read; cache in memory per service	Avoids DB hammering; acceptable for MVP scale
Display board refresh	SSE from same endpoint as citizen tracking	Reuses SSE infrastructure